系统设计文档

数据备份软件（DataBackup）

组长: 王科瑾
学号: 待填写

目录

系统设计文档.....1
1. 开发环境和工具.....1
1.1 开发环境.....1
1.2 依赖库.....2
2. 总体设计.....3
2.1 系统结构设计.....3
3. 静态建模.....7
3.1 系统对象模型.....7
3.2 类（对象）描述.....8
3.3 其它静态建模方式（可选）.....21
4. 动态建模.....22
4.1 系统场景.....22
4.2 系统状态.....25
5. 数据库设计（如有）.....27

系统设计文档

1. 开发环境和工具

1.1 开发环境

（1）硬件平台：x86-64 个人计算机。

（2）操作系统：开发主机 Windows + Docker Desktop；**构建/测试/演示均在容器或 WSL2 中的 Ubuntu 22.04/24.04**（目标平台 Linux，与课程要求一致）。

（3）编译环境：Go 1.27.1（`go build`/`go test`/`gofmt`/`staticcheck` 2026.2.1）、Node.js 22.23.3 + npm（electron-vite 构建）、GNU Make；统一封装在 `docker/dev.Dockerfile` 开发镜像内（版本由 `scripts/install-deps.sh` 锁定并 SHA256 校验），`bash scripts/docker.sh check` 一键门禁，CI 跑在 GitHub ubuntu-22.04 runner 上，使用同一依赖安装脚本与门禁命令。

（4）IDE：Windsurf（配合 AI 编码 agent）；本项目使用 AI 辅助开发。

1.2 依赖库

后端：仅 Go 标准库，零第三方生产依赖。归档（MyPack）、压缩（Huffman）、密码学（SHA-256、HMAC、PBKDF2、AES-256-CBC）全部自研——课程规定第三方实现的扩展功能减半记分；`scripts/check-imports.sh` 在 lint 阶段强制拦截 `archive/*`、`compress/*`、`crypto/aes|cipher|des|sha*|md5|hmac|pbkdf2`、`golang.org/x/*` 等导入。唯一允许的测试依赖是 `github.com/rogpeppe/go-internal/testscript`（CLI 黑盒测试）。

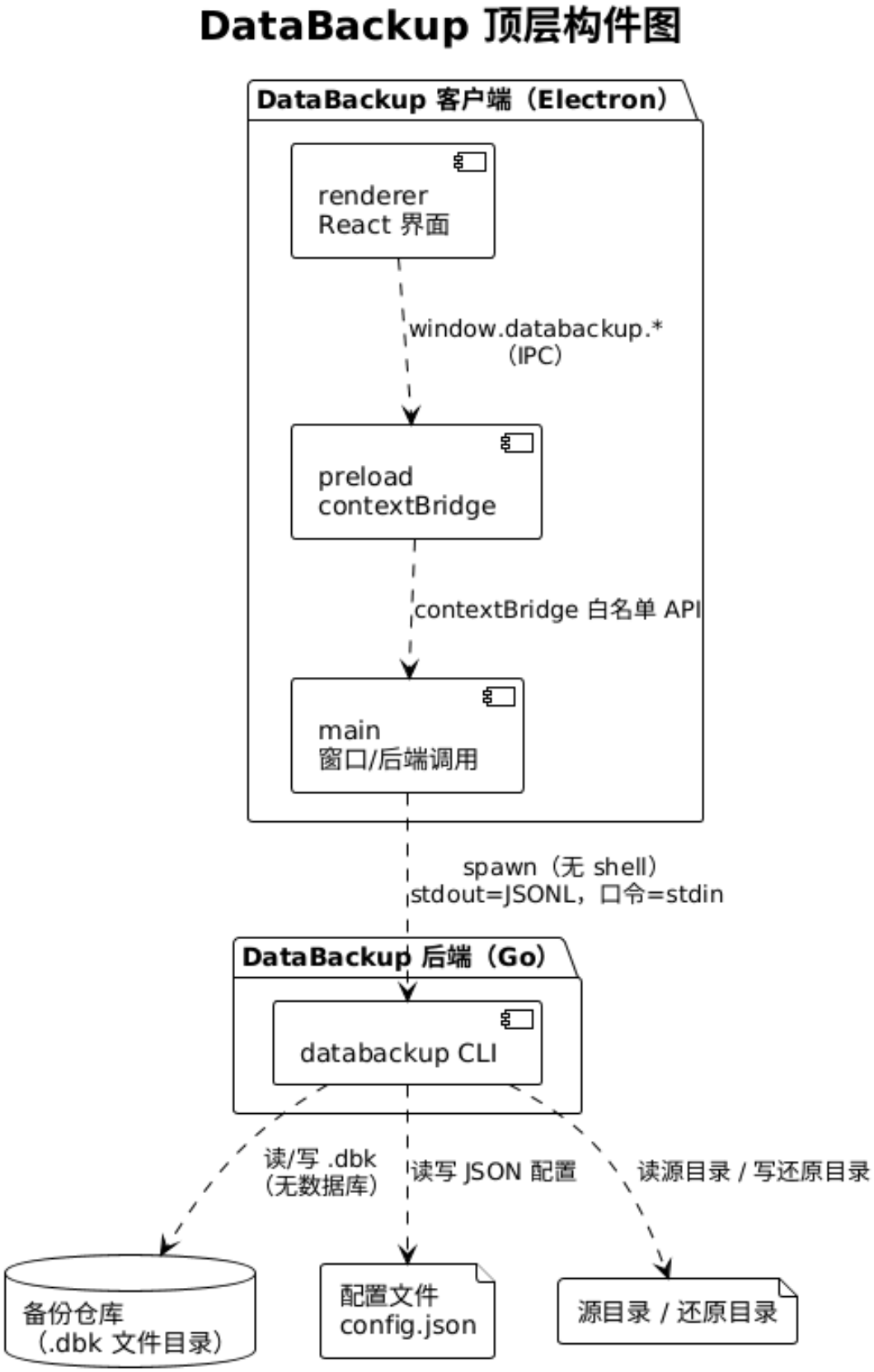
客户端：Electron 44、React 19、TypeScript 5.9、electron-vite 5、electron-builder 26（打包 AppImage/deb）；界面库 Tailwind v4 + shadcn/ui 及配套（zustand、zod、sonner 等，版本在 `client/package.json` 精确锁定）；测试 Vitest 5 + Playwright 1.64。界面库不参与备份逻辑——所有备份/还原/过滤/压缩/加密均由 Go 后端完成，客户端只做界面与进程调用。

2. 总体设计

2.1 系统结构设计

2.1.1 顶层系统结构

2.1.1.1 构件图



（源文件 `diagrams/component-top.puml`。）

2.1.1.2 构件图描述

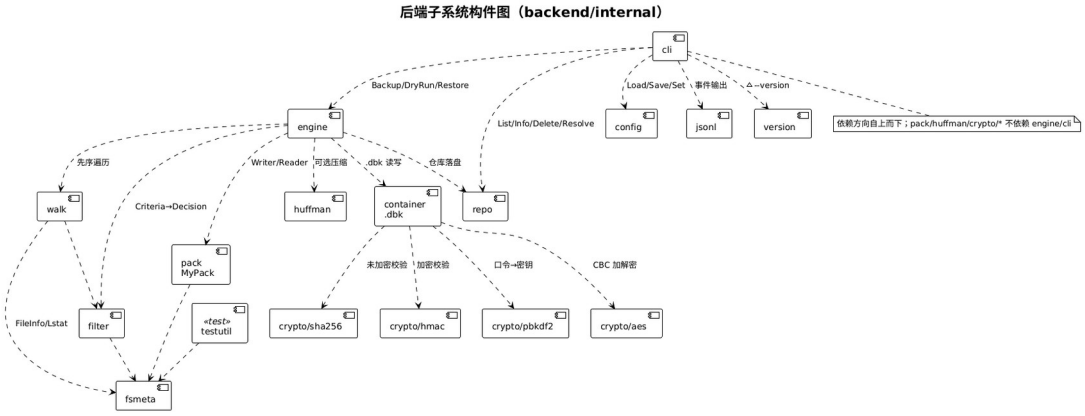
系统由两个可独立部署的构件组成:

- **客户端（Electron）**：renderer（React 界面）只经 `preload` 暴露的 `window.databackup` 白名单 API 与 `main` 进程通信；`main` 负责窗口管理与后端调用，以 `spawn`（无 shell）启动 `databackup` 子进程，stdout 按行解析 JSONL 事件，口令仅经 stdin 传递。安全三件套 `contextIsolation/sandbox/nodeIntegration=false` 不变。
- **后端（Go）**：`databackup` CLI 单二进制，读写备份仓库（`.dbk` 文件目录）、配置文件与用户文件系统。**系统不使用数据库**（ADR-005）：备份列表与详情由扫描 `.dbk` 文件头尾得出，仓库即文件系统目录。

两构件间的唯一接口是 `../architecture/cli-contract.md` 冻结的 CLI/JSONL 协议，两端各有契约测试比对协议样例。

2.1.2 子系统一结构（后端核心）

2.1.2.1 构件图



（源文件 `diagrams/component-backend.puml`。）

2.1.2.2 构件图描述

后端按数据流水线分层，依赖方向自上而下（`docs/architecture/overview.md` §2）：

- **入口层**：`cli`（子命令/参数/输出/错误映射）→ `jsonl`（协议类型与 `Emitter`）、`version`。
- **编排层**：`engine` 编排 `walk → filter → pack → [huffman] → container(→[crypto])` 的备份流水线及其逆过程的还原流水线；进度节流在 engine 完成（每阶段 ≤10 次/秒，阶段结束必发一次）。
- **原语层**：`fsmeta`（文件信息模型与 Linux 元数据读写）、`walk`（先序遍历+硬链接识别）、`filter`（Include/Exclude/SkipDir 判定）、`pack`（MyPack 归档）、`huffman`（分块规范

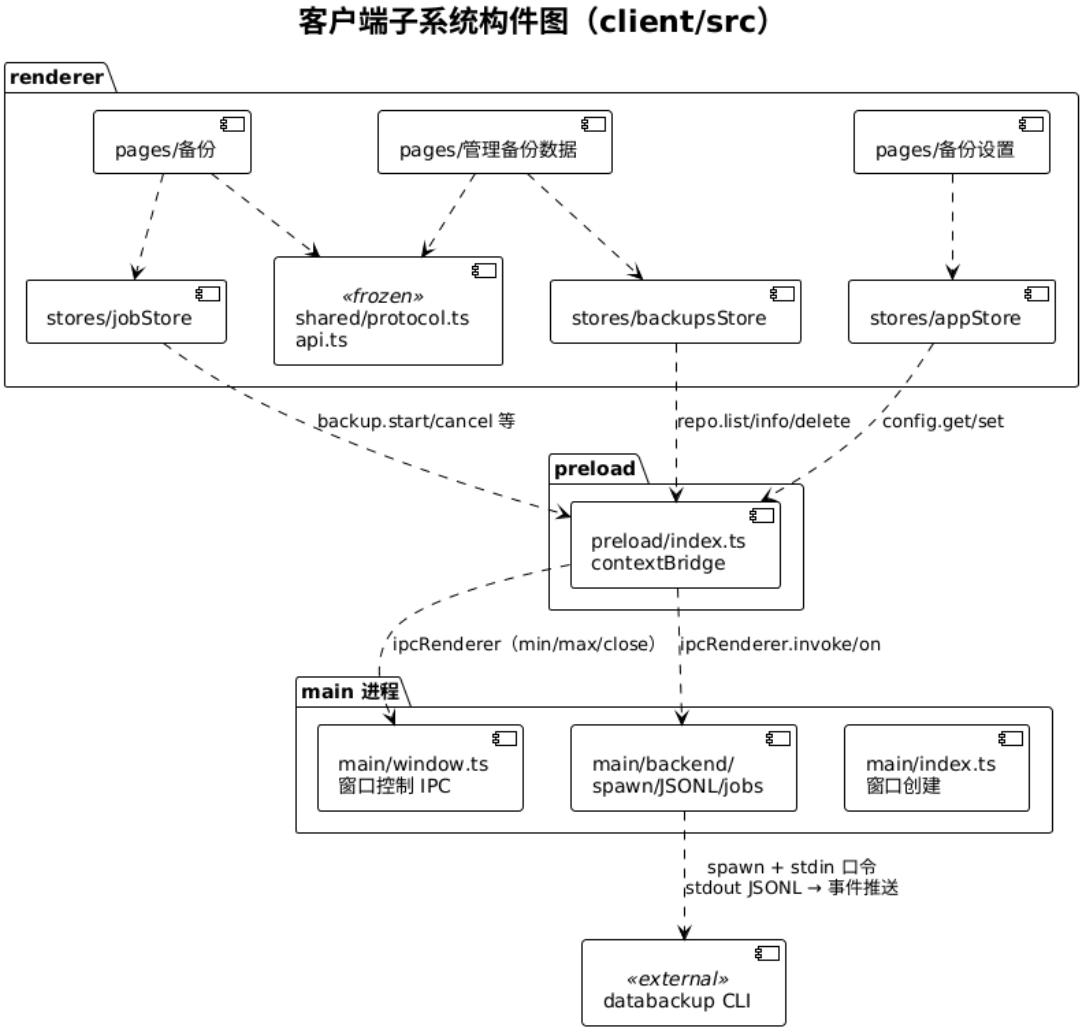
Huffman）、`crypto/sha256|hmac|pbkdf2|aes`（手写密码学原语）、`container`（`.dbk` 容器：明文头+负载+尾部统计+校验值）。

- **支撑层**：`repo`（仓库扫描/查看/删除/名称规则）、`config`（`config.json` 读写）、`testutil`（测试夹具）。

原语包不得依赖 `engine`/`cli`，可独立测试与并行开发；面向用户的错误一律为包级哨兵错误，经 `cli.Classify` 映射为契约 §4 的错误码与退出码。

2.1.3 子系统二结构（客户端）

2.1.3.1 构件图



（源文件 `diagrams/component-client.puml`。）

2.1.3.2 构件图描述

客户端三进程结构（`docs/client/interaction.md` §2）：

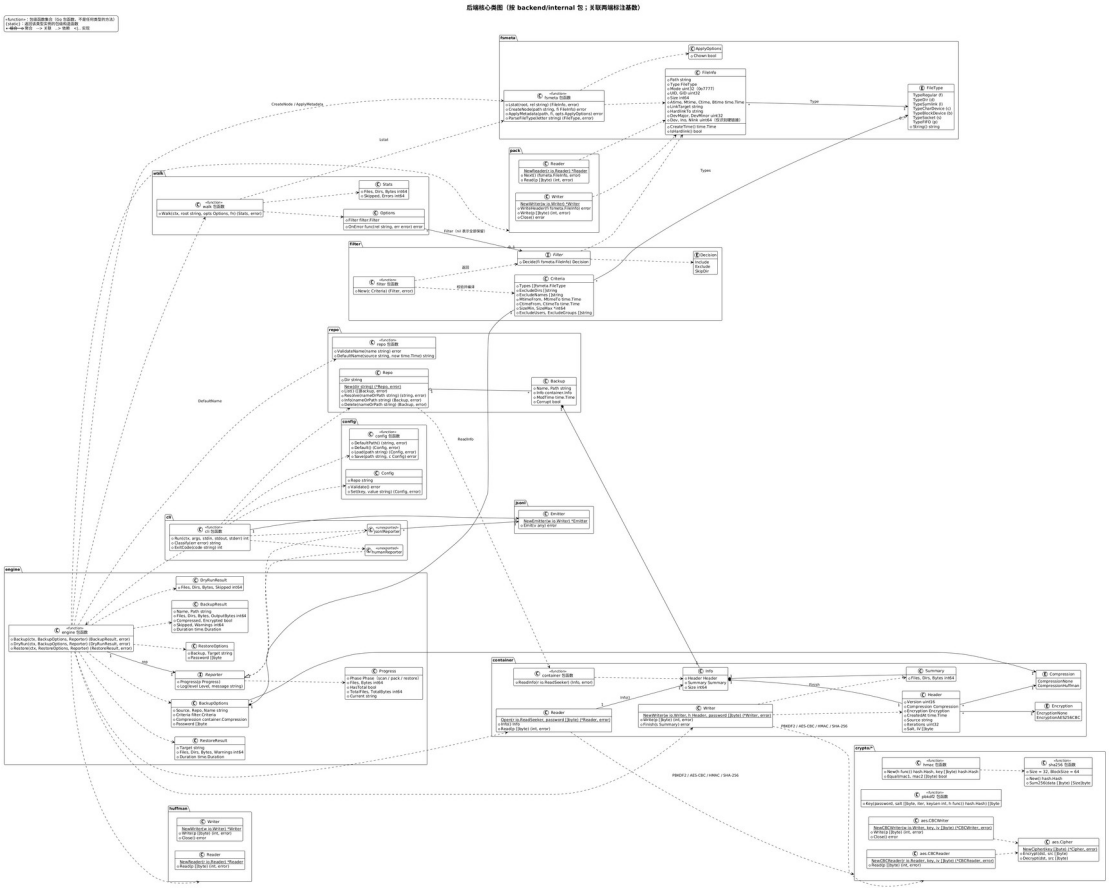
- **main**：`index.ts` 创建无边框窗口；`window.ts` 提供最小化/最大化/关闭 IPC 与最大化状态推送；`backend/` 目录维护后端调用——

`runner.ts` spawn 子进程并逐行解析 JSONL（口令写 stdin 后立即关闭、取消先 SIGTERM 3 秒后 SIGKILL）、`jobs.ts` 任务注册表（备份/还原互斥 `E_BUSY`）、`args.ts`/`resolve.ts`/`jsonl.ts`/`mock.ts` 等，转为 IPC 事件推给 renderer；`ipc/` 内 zod 校验的处理器与 `dialogs.ts` 目录选择/关闭确认。

- **preload**：`contextBridge` 暴露 `window.databackup`（应用/窗口/对话框/备份/还原/任务/仓库/配置八个命名空间），方法签名与 `src/shared/api.ts` 冻结类型一致。
- **renderer**：zustand 三个 store——`jobStore`（备份/还原任务状态机，见 §4.2）、`backupsStore`（列表缓存）、`appStore`（页面导航、后端状态与版本、减少动效、侧栏折叠）；三个页面——备份、管理备份数据、设置；`src/shared/` 协议与 API 类型为冻结件，与后端 `jsonl` 类型经契约测试保持一致。

3. 静态建模

3.1 系统对象模型



（源文件 `diagrams/class-backend.puml`。按包组织；关联两端标注基数（`1`、`0..1`、`*`、`0..*`），`*--` 为组合、`o--` 为聚合、`-->` 为关联、`..>` 为依赖、`<|..` 为接口实现。Go 的包级函数不属于任何类型，图中以 `<<function>>` 伪

类汇总；返回某类型实例的包级构造函数（如 pack.NewWriter、container.Open）以 {static} 标在该类型内。主要关系：Repo "1" o-- "*" Backup（仓库聚合多个备份）；Backup "1" *-- "1" Info、Info "1" *-- "1" Header、Info "1" *-- "1" Summary（容器信息按值组合）；BackupOptions "1" *-- "1" Criteria；walk.Options "*" --> "0..1" Filter（为 nil 时全部保留）；FileInfo "*" --> "1" FileType；cli 中两个未导出类型 jsonlReporter、humanReporter 实现 engine.Reporter。）

3.2 类（对象）描述

以下类描述与 backend/internal/* 各包已冻结的公开 API（go doc ./internal/<包>）一一对应；签名照抄源码，ctx 均为 context.Context。"包函数"指 Go 包级函数，不是某个类型的方法。接口 engine.Reporter 由 cli 实现适配到 JSONL 事件或人类可读输出。

3.2.1 fsmeta.FileInfo 类（对象）

3.2.1.1 类描述

文件信息模型：walk → filter → pack → 还原全链路共用的数据模型，覆盖 Linux 全部文件类型与需要恢复的元数据。M1-3 冻结，之后只能新增字段。硬链接不是独立类型，而是 Type 为 TypeRegular 且 HardlinkTo 非空的条目。配套类型：FileType（uint8 枚举，TypeRegular…TypeFIFO 七个取值，String() 返回 --types 字母 f d l c b s p）、ApplyOptions（控制 ApplyMetadata 恢复哪些元数据）。

3.2.1.2 属性描述

属性	类型	说明
Path	string	相对源目录的路径，/ 分隔，不含 .、.. 段，不以 / 开头或结尾
Type	FileType	文件类型：f 普通 d 目录 l 软链接 c 字符设备 b 块设备 s 套接字 pFIFO
Mode	uint32	权限位及 setuid/setgid/sticky 位（低 12 位，即 0o7777），不含类型位
UID, GID	uint32	数字属主与属组

属性	类型	说明
Size	int64	普通文件的数据长度（字节）；其他类型以及硬链接条目为 0
Atime, Mtime, Ctime	time.Time	访问、修改、状态变更时间（纳秒精度）
Btime	time.Time	创建时间（statx 的 btime）；文件系统不支持时为零值
LinkTarget	string	软链接内容（原样保存，不解析）；其他类型为空
HardlinkTo	string	非空表示本条目是硬链接，值为归档中更早出现的同一 inode 条目的 Path；此时不携带数据
DevMajor, DevMinor	uint32	字符/块设备的主、次设备号；其他类型为 0
Dev, Ino, Nlink	uint64	源文件系统上的设备号、inode 号与链接数，**只用于遍历时识别硬链接，不写入归档**
`ApplyOptions.Chown`	bool	为 true 时恢复 UID/GID（需要 root）；非 root 时调用方置 false 并发 warn

3.2.1.3 方法描述

方法 / 包函数	说明
`(fi FileInfo) CreateTime() time.Time`	方法：返回"创建时间"过滤所用的时间，有 Btime 用 Btime，否则退化为 Ctime
`(fi FileInfo) IsHardlink() bool`	方法：`HardlinkTo` 非空时为 true
`(t FileType) String() string`	方法：返回 `--types` 字母，非法值返回 `?`

方法 / 包函数	说明
`ParseFileType(letter string) (FileType, error)`	包函数：把 `--types` 中的一个字母解析为 FileType
`Lstat(root, rel string) (FileInfo, error)`	包函数（Linux）：读取 `root/rel` 的信息，不跟随软链接；优先用 statx 读 btime；不填 `HardlinkTo`
`CreateNode(path string, fi FileInfo) error`	包函数（Linux）：创建特殊节点——FIFO（mkfifo）、字符/块设备与套接字（mknod）；普通文件、目录、软/硬链接不由它创建，也不设置元数据
`ApplyMetadata(path string, fi FileInfo, opts ApplyOptions) error`	包函数（Linux）：按需 lchown，再 chmod（软链接跳过），最后设置 atime/mtime（软链接设置链接本身）；Ctime、Btime 无法设置；目录应在其子项全部还原后调用

3.2.2 filter.Criteria / filter.Filter 类

3.2.2.1 类描述

`Criteria` 是用户给出的过滤条件，字段与 CLI 过滤参数一一对应，零值表示不过滤。`Filter` 是**接口**，由包函数 `New(Criteria)` 校验并编译条件后返回（内部实现为未导出的 `compiledFilter`，并发安全的纯函数），对每个 `FileInfo` 给出 `Decision`：`Include`（记录；目录还会继续进入）、`Exclude`（不记录；目录仍进入其子项）、`SkipDir`（只用于目录：不记录也不进入）。语义以 `cli-contract.md` §2.1"过滤语义"为准。

3.2.2.2 属性描述

属性	类型	说明
Types	[]fsmeta.FileType	只备份所列类型；空表示全部
ExcludeDirs	[]string	要排除的目录，相对源目录、`/` 分隔（绝对路径已由调用方换算）
ExcludeNames	[]string	正则（RE2）列表，文件名（不含目录部分）匹配任一则排除
MtimeFrom,	time.Time	修改时间闭区间，零值

属性	类型	说明
MtimeTo		表示该端不限
CtimeFrom, CtimeTo	time.Time	创建时间闭区间（使用 `FileInfo.CreateTime()`），零值不限
SizeMin, SizeMax	`*int64`	尺寸闭区间（字节），只作用于普通文件；nil 表示该端不限
ExcludeUsers, ExcludeGroups	[]string	排除的属主/属组，名称或十进制数字 ID

3.2.2.3 **方法描述**

方法 / 包函数	说明
`New(c Criteria) (Filter, error)`	包函数：编译正则、解析用户/组名为 ID、检查区间；非法（未知类型、非法排除路径、正则错误、区间颠倒、负尺寸、用户/组不存在）返回包装 `ErrInvalidCriteria` 的错误（cli 映射 E_USAGE）
`Decide(fi fsmeta.FileInfo) Decision`	接口方法：排除目录命中 → SkipDir；其他目录只由 Types 决定 Include/Exclude；非目录须同时通过类型、名字、时间、尺寸（仅普通文件）、用户/组条件

3.2.3 walk.Options / walk.Stats 类

3.2.3.1 **类描述**

先序遍历源目录，产出经过过滤的 `fsmeta.FileInfo` 序列。约定：不跟随软链接（lstat）；同一目录内按名字字节序排序，输出顺序稳定；源目录本身不输出；按 Dev+Ino 识别硬链接（只对被保留的普通文件），第二次及以后出现的路径填写 `HardlinkTo`。进度节流由 engine 负责，本包不做。

3.2.3.2 **属性描述**

类	属性	类型	说明
Options	Filter	filter.Filter	过滤器（接口）；nil 时全部保留
Options	OnError	`func(rel string, err`	处理单个条目的读取错误：返回

类	属性	类型	说明
		`error) error`	nil 跳过该条目继续，返回非 nil 中止遍历；为 nil 时任何错误都中止
Stats	Files	int64	保留的非目录条目数（含链接与特殊文件）
Stats	Dirs	int64	保留的目录数
Stats	Bytes	int64	保留的普通文件数据总字节（硬链接条目不计）
Stats	Skipped	int64	被过滤掉的条目数（SkipDir 的目录计 1，不计其内部）
Stats	Errors	int64	经 OnError 跳过的条目数

3.2.3.3 方法描述

包函数	说明
`Walk(ctx, root string, opts Options, fn func(fsmeta.FileInfo) error) (Stats, error)`	先序遍历 root，对每个保留的条目调用 fn；fn 返回错误时中止并返回该错误；ctx 取消时尽快返回 `ctx.Err()`

3.2.4 pack.Writer / pack.Reader 类

3.2.4.1 类描述

MyPack 归档的写/读器，用法同 `archive/tar`：每个对象是 `BlockSize`（512 字节，来源 tar/UStar）的头记录加补齐到 512 倍数的数据；长文件名使用多个头块；归档以两个全零块结尾。`Reader` 除拒绝绝对路径与 `..` 段外，还拒绝以已出现的软链接为祖先的路径。字节级格式见 `../architecture/mypack-format.md`。

3.2.4.2 属性描述

无公开字段；写/读状态内部维护（当前条目、剩余字节）。包级常量与哨兵错误：`BlockSize = 512`；`Magic = "MYPK"`（每个头块开头的魔数）；`Version = 1`（当前格式版本）；`MaxNameLen = 1 << 16`（路径、软链接目

标、硬链接目标的最大字节数）；`ErrCorrupt`（魔数/校验和不符、字段非法、意外截断，cli 映射 E_CORRUPT）；`ErrUnsafePath`（路径为绝对路径、含 `..` 或为空，E_CORRUPT）；`ErrWriteTooLong`（写入数据超过头记录声明的 Size）。

3.2.4.3 方法描述

方法 / 包函数	说明
`NewWriter(w io.Writer) *Writer`	包函数：返回写入 w 的 Writer
`(w *Writer) WriteHeader(fi fsmeta.FileInfo) error`	写一个条目的头记录（必要时多个头块）；普通文件（非硬链接）之后须恰好写入 `fi.Size` 字节；上一条目数据不足时报错；Path 不合法返回 `ErrUnsafePath`
`(w *Writer) Write(p []byte) (int, error)`	写当前条目数据；超过声明长度时只写到声明长度并返回 `ErrWriteTooLong`
`(w *Writer) Close() error`	补齐最后一个条目、写入结尾两个零块；不关闭底层 Writer；重复调用返回 nil
`NewReader(r io.Reader) *Reader`	包函数：返回从 r 读取的 Reader
`(r *Reader) Next() (fsmeta.FileInfo, error)`	跳过当前条目剩余数据、读下一条目头；结尾零块返回 `io.EOF`；校验失败 `ErrCorrupt`，路径不安全 `ErrUnsafePath`；返回的 Dev、Ino、Nlink 为 0
`(r *Reader) Read(p []byte) (int, error)`	读当前条目数据，读完返回 `io.EOF`

3.2.5 huffman.Writer / huffman.Reader 类

3.2.5.1 类描述

分块流式 Huffman 编解码：输入按 `BlockSize`（1 MiB）分块，每块统计频率、构造规范 Huffman 码表并编码，块头记录 256 项码长表；流以 4 字节全零的结束块结尾。`Reader` 对一切非法输入返回包装 `ErrCorrupt` 的错误，不 panic、不死循环。

3.2.5.2 属性描述

写/读器无公开字段。包级常量与哨兵错误：`BlockSize = 1 << 20`（每块最大原始长度）；`ErrCorrupt`（码表非法、截断等，cli 映射 E_CORRUPT）。

3.2.5.3 方法描述

方法 / 包函数	说明
`NewWriter(w io.Writer) *Writer`	包函数：返回把压缩结果写入 w 的 Writer
`(w *Writer) Write(p []byte) (int, error)`	缓冲数据，每满一块编码输出一次
`(w *Writer) Close() error`	编码剩余数据并写入流结束标记；不关闭底层 Writer
`NewReader(r io.Reader) *Reader`	包函数：返回从 r 读取压缩流并输出原始数据的 Reader
`(r *Reader) Read(p []byte) (int, error)`	逐块解压；读到结束块返回 `io.EOF`，格式错误返回 `ErrCorrupt`

3.2.6 crypto/sha256、crypto/hmac、crypto/pbkdf2、crypto/aes 类

3.2.6.1 类描述

手写密码学原语，签名仿标准库，以便在 `_test.go` 中与标准库对照（标准库只允许出现在测试中）。`sha256` 实现 FIPS 180-4；`hmac` 实现 RFC 2104；`pbkdf2` 实现 RFC 8018 §5.2，用于从口令派生 AES 与 HMAC 密钥；`aes` 实现 AES-256 分组密码（FIPS-197）以及 CBC 模式 + PKCS#7 填充的流式加解密（NIST SP 800-38A），不提供其他密钥长度与模式。前三个包只有包函数；`aes` 有 `Cipher`、`CBCWriter`、`CBCReader` 三个类型（均无公开字段）。

3.2.6.2 属性描述

包	常量 / 哨兵错误	说明
sha256	`Size = 32`	摘要长度（字节）
sha256	`BlockSize = 64`	分组长度（字节）
aes	`BlockSize = 16`	AES 分组长度（字节），也是 IV 长度
aes	`KeySize = 32`	AES-256 密钥长度（字节）
aes	`ErrKeySize`	密钥不是 32 字节
aes	`ErrIVSize`	IV 不是 16 字节
aes	`ErrPadding`	解密后 PKCS#7 填充非法或密文长度不是分组整数倍（cli 映射 E_CORRUPT）

3.2.6.3 方法描述

方法 / 包函数	说明
`sha256.New() hash.Hash`	包函数：返回计算 SHA-256 的 `hash.Hash`
`sha256.Sum256(data []byte) [Size]byte`	包函数：一次性计算 data 的摘要
`hmac.New(h func() hash.Hash, key []byte) hash.Hash`	包函数：以 h 为底层哈希、key 为密钥的 HMAC
`hmac.Equal(mac1, mac2 []byte) bool`	包函数：常量时间比较两个 MAC，长度不同直接返回 false
`pbkdf2.Key(password, salt []byte, iter, keyLen int, h func() hash.Hash) []byte`	包函数：用 HMAC(h) 从 password、salt 迭代 iter 次派生 keyLen 字节密钥；iter ⩽ 0、keyLen ⩽ 0 或 keyLen 超过 (2^32-1)·h().Size() 属于调用方编程错误，直接 panic
`aes.NewCipher(key []byte) (*Cipher, error)`	包函数：密钥扩展；key 不是 32 字节返回 `ErrKeySize`
`(c *Cipher) Encrypt(dst, src []byte)` / `Decrypt(dst, src []byte)`	加密/解密 src 的第一个分组写入 dst（可原地）；`Cipher` 可并发使用
`aes.NewCBCWriter(w io.Writer, key, iv []byte) (*CBCWriter, error)`	包函数：返回把 CBC 密文写入 w 的写入器
`(w *CBCWriter) Write(p []byte) (int, error)` / `Close() error`	Write 加密并写出所有完整分组，不足一组留待后续；Close 写出带 PKCS#7 填充的最后一组，不关闭底层 Writer
`aes.NewCBCReader(r io.Reader, key, iv []byte) (*CBCReader, error)`	包函数：返回从 r 读取密文的解密读取器
`(r *CBCReader) Read(p []byte) (int, error)`	读取明文；流结束返回 `io.EOF`，在流末校验并去除填充，非法返回 `ErrPadding`

3.2.7 container 相关类（Header / Summary / Info / Writer / Reader）

3.2.7.1 类描述

`.dbk` 容器格式：明文头部（魔数、版本、算法、创建时间、源目录、KDF 参数、盐、IV）+ 负载（MyPack 流，可选 Huffman，加密时为 AES-256-CBC 密文）+ 明文尾部（`Summary` 统计）+ 校验值（加密：PBKDF2-HMAC-

SHA256 派生 64 字节，前 32 字节为 AES 密钥、后 32 字节为 HMAC 密钥，校验值 = HMAC-SHA256(头部 ‖ 密文 ‖ 尾部)；不加密：SHA-256(头部 ‖ 负载 ‖ 尾部)）。`Open` 先完整校验再允许读取；`ReadInfo` 只读头尾用于 list/info，无需口令。

3.2.7.2 属性描述

类	属性	说明
Header	`Version uint16`	格式版本：读取时填写；写入时忽略，总写 `FormatVersion`（=1）
Header	`Compression Compression`、`Encryption Encryption`	负载算法：`CompressionNone`/`CompressionHuffman`；`EncryptionNone`/`EncryptionAES256CBC`（均为 `uint8` 枚举）
Header	`CreatedAt time.Time`、`Source string`	创建时间（秒精度保存）、源目录绝对路径
Header	`Iterations uint32`、`Salt, IV []byte`	只在加密时有意义：写入时由 `NewWriter` 生成（Iterations 为 0 时用 `DefaultIterations` = 100000），读取时填写
Summary	`Files, Dirs, Bytes int64`	写在尾部的统计，口径同 backup 结果
Info	`Header Header`、`Summary Summary`、`Size int64`	不校验、不解密即可读到的信息；Size 为整个文件字节数
（包级）	`Ext = ".dbk"`、`PartialExt = ".partial"`	备份文件扩展名；写入中的临时文件为 `<名称>.dbk.partial`
（包级）	`ErrAuth`、`ErrPasswordRequired`、`ErrCorrupt`、`ErrUnsupportedVersion`	分别映射 E_AUTH、E_PASSWORD_REQUIRED、E_CORRUPT、E_COR

类	属性	说明
		RUPT

3.2.7.3 方法描述

方法 / 包函数	说明
`NewWriter(w io.Writer, h Header, password []byte) (*Writer, error)`	包函数：写出头部并返回负载写入器；`h.Encryption` 非 None 时 password 必须非空，由本函数用 `crypto/rand` 生成盐与 IV 并派生密钥；为 None 时 password 必须为 nil
`(w *Writer) Write(p []byte) (int, error)`	写入负载（加密时边写边加密、边计算 HMAC）
`(w *Writer) Finish(s Summary) error`	结束负载，写出尾部 s 与校验值；不关闭底层 Writer
`ReadInfo(r io.ReadSeeker) (Info, error)`	包函数：只读头部与尾部（不校验、不需要口令）；格式错误返回 `ErrCorrupt` 或 `ErrUnsupportedVersion`
`Open(r io.ReadSeeker, password []byte) (*Reader, error)`	包函数：先完整读一遍 r 校验（加密：HMAC，失败 `ErrAuth`；未加密：SHA-256，失败 `ErrCorrupt`），通过后回到负载起点返回明文读取器；加密备份 password 为空返回 `ErrPasswordRequired`；未加密备份忽略 password
`(r *Reader) Info() Info`	返回容器的头部、尾部与文件大小
`(r *Reader) Read(p []byte) (int, error)`	读取负载明文（仍是打包流，可能已压缩），读完返回 `io.EOF`

3.2.8 engine 相关类（BackupOptions / RestoreOptions / Reporter）

3.2.8.1 类描述

编排层（只属于 W3 集成泳道）：包函数 `Backup`/`DryRun`/`Restore` 把选项对象与 `Reporter` 串联到流水线。`Reporter` 是接口（进度+日志），由 `cli` 中未导出的 `jsonlReporter`（JSONL 事件）与 `humanReporter`（stderr 文本）实现。面向用户的错误类别为包级哨兵错误：`ErrSourceNotFound`（E_NOT_FOUND）、`ErrSourceNotDir`、`ErrInvali`

dOptions（E_USAGE）、ErrExists（E_EXISTS）、ErrBackupNotFound（E_NOT_FOUND）、ErrTargetNotEmpty（E_TARGET_NOT_EMPTY）；下层包的哨兵错误原样包装上传，ctx 取消返回 context.Canceled（E_CANCELLED）。

3.2.8.2 属性描述

类	属性	说明
BackupOptions	Source, Repo, Name string、Criteria filter.Criteria、Compression container.Compression、Password []byte	Repo 不存在则创建；Name 空时用 repo.DefaultName；Criteria.ExcludeDirs 可为源内绝对路径，由 engine 换算；Password 非 nil 表示 AES-256-CBC，engine 不保留其引用
RestoreOptions	Backup, Target string、Password []byte	Backup 为已经 repo.Resolve 解析的路径；Target 必须不存在或为空；备份未加密却给了 Password 时发 warn 并忽略
BackupResult	Name, Path string、Files, Dirs, Bytes, OutputBytes int64、Compressed, Encrypted bool、Skipped, Warnings int64、Duration time.Duration	对应契约 backup result
DryRunResult	Files, Dirs, Bytes, Skipped int64	对应契约 dry-run result
RestoreResult	Target string、Files, Dirs, Bytes, Warnings int64、Duration time.Duration	对应契约 restore result

类	属性	说明
Progress	`Phase` `Phase、Files, Bytes` `int64、HasTotal` `bool、TotalFiles, TotalBytes` `int64、Current` `string`	Phase ∈ `PhaseScan/PhasePack/PhaseRestore`（"scan"/"pack"/"restore"）；HasTotal 为 true 时（pack、restore 阶段）总量有效
Level	`LevelInfo / LevelWarn / LevelError`	日志级别（int 枚举）

3.2.8.3 方法描述

方法 / 包函数	说明
`Backup(ctx, opts BackupOptions, rep Reporter) (BackupResult, error)`	包函数：先 scan 得总量，再 pack 一次流式写出 `.partial`，成功后经 `os.Link` 硬链接原子发布为 `.dbk`（遇同名 `E_EXISTS`，防并发覆盖），随后删除 `.partial`；失败或取消删除 `.partial`
`DryRun(ctx, opts BackupOptions, rep Reporter) (DryRunResult, error)`	包函数：只遍历与过滤，不写文件；忽略 Name、Compression、Password
`Restore(ctx, opts RestoreOptions, rep Reporter) (RestoreResult, error)`	包函数：先完整校验备份再写出文件；取消时不回滚，发 warn 说明目标不完整
`Reporter.Progress(p Progress)` / `Reporter.Log(level Level, message string)`	接口方法：进度与日志回调；engine 在同一 goroutine 中调用，Progress 已节流（每阶段 ≤10 次/秒，阶段结束必发一次）

3.2.9 repo.Repo / repo.Backup 类

3.2.9.1 类描述

仓库目录对象：扫描 `.dbk` 文件、读容器头尾得列表（不使用数据库或索引文件，ADR-005）；名称↔路径解析；删除只允许 `.dbk`。哨兵错误：`ErrNotFound`（E_NOT_FOUND）、`ErrInvalidName`（备份名不合法或删除目标不是 `.dbk`，E_USAGE）。

3.2.9.2 属性描述

属性	类型	说明
Repo.Dir	string	仓库目录绝对路径
Backup.Name、Backup.Path	string	文件名去掉 .dbk / 绝对路径
Backup.Info	container.Info	容器头尾信息（Corrupt 时为零值）
Backup.ModTime	time.Time	文件修改时间（损坏备份代替创建时间展示）
Backup.Corrupt	bool	头部或尾部无法解析

3.2.9.3 方法描述

方法 / 包函数	说明
New(dir string) (*Repo, error)	包函数：返回位于 dir 的仓库（转为绝对路径；不要求已存在）
(r *Repo) List() ([]Backup, error)	列出全部 .dbk（忽略 .partial 与其他文件），损坏的以 Corrupt 列出；按创建时间降序、名字升序；目录不存在时返回空切片
(r *Repo) Resolve(nameOrPath string) (string, error)	含 / 视为路径，否则视为仓库内名称（可带或不带 .dbk）；不检查文件是否存在
(r *Repo) Info(nameOrPath string) (Backup, error)	读一个备份的信息；不存在 ErrNotFound，损坏 container.ErrCorrupt
(r *Repo) Delete(nameOrPath string) (Backup, error)	删除一个 .dbk 并返回其信息（损坏的也可删除）
ValidateName(name string) error	包函数：1~200 字节，不含 / 与 NUL，不以 . 开头
DefaultName(source string, now time.Time) string	包函数：<源目录名>-<YYYYMMDD-HHMMSS>（本地时间），源为 / 时记作 root；源目录名以 . 开头去掉全部前导点（去空记 root），结果超 200 字节在 UTF-8 字符边界截短源目录名部分

3.2.10 config.Config 类

3.2.10.1 类描述

用户配置（当前仅 repo，`Keys = []string{"repo"}`），JSON 存于 `$XDG_CONFIG_HOME/databackup/config.json`（未设置时 `~/.config/databackup/config.json`）；`Save` 经同目录临时文件原子改名，并发写不损坏文件。哨兵错误 `ErrInvalid`（配置值非法或 KEY 未知，E_USAGE）。

3.2.10.2 属性描述

属性	类型	说明
Repo	string	备份仓库目录绝对路径（JSON key `repo`）

3.2.10.3 方法描述

方法 / 包函数	说明
`DefaultPath() (string, error)` / `Default() (Config, error)`	包函数：配置文件路径（XDG_CONFIG_HOME → HOME/.config）/ 默认配置（Repo 为 `~/DataBackup`）
`Load(path string) (Config, error)`	包函数：文件不存在返回 `Default()`，缺失字段用默认值补齐；非法 JSON 或非法值返回包装 `ErrInvalid` 的错误
`Save(path string, c Config) error`	包函数：校验后写同目录临时文件再原子改名，必要时创建父目录
`(c Config) Validate() error`	仓库路径须为绝对路径；已存在时必须是目录
`(c Config) Set(key, value string) (Config, error)`	把 KEY=VALUE 写入副本并校验，返回新配置；KEY 未知返回 `ErrInvalid`

3.3 其它静态建模方式（可选）

对象关系补充见 §2.1 构件图与 §5 的 `.dbk` 容器格式字段表；客户端类型关系由冻结的 `client/src/shared/protocol.ts`、`api.ts` 定义并经契约测试约束，不再另绘。

4. 动态建模

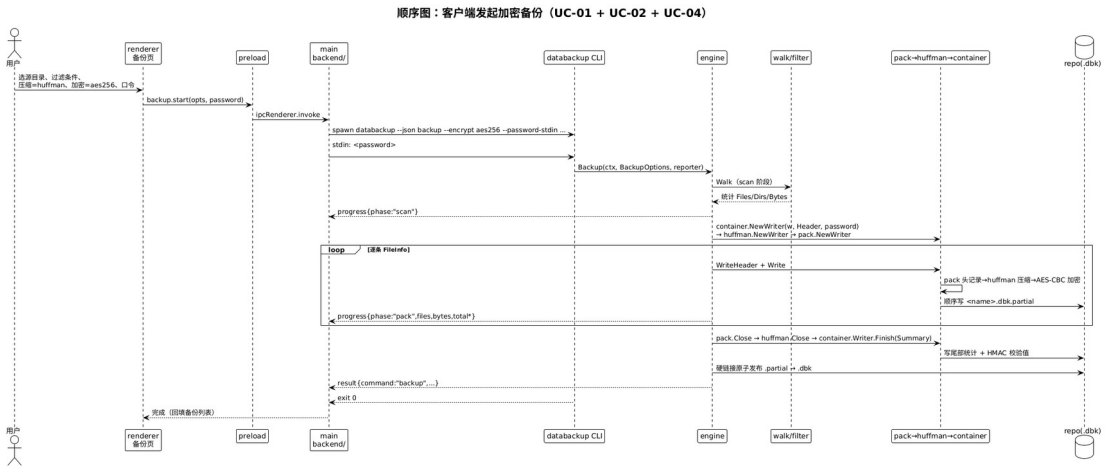
4.1 系统场景

4.1.1 场景：客户端发起加密备份（含过滤/压缩/加密）

4.1.1.1 场景描述

用户在"备份"页选择源目录、过滤条件、Huffman 压缩与 AES-256 加密并输入口令；客户端 spawn `databackup backup`，进度经 JSONL 实时回显，成功后仓库新增 `.dbk`（UC-01/02/04）。

4.1.1.2 顺序图



（源文件 `diagrams/seq-backup.puml`。）

4.1.1.3 流程说明

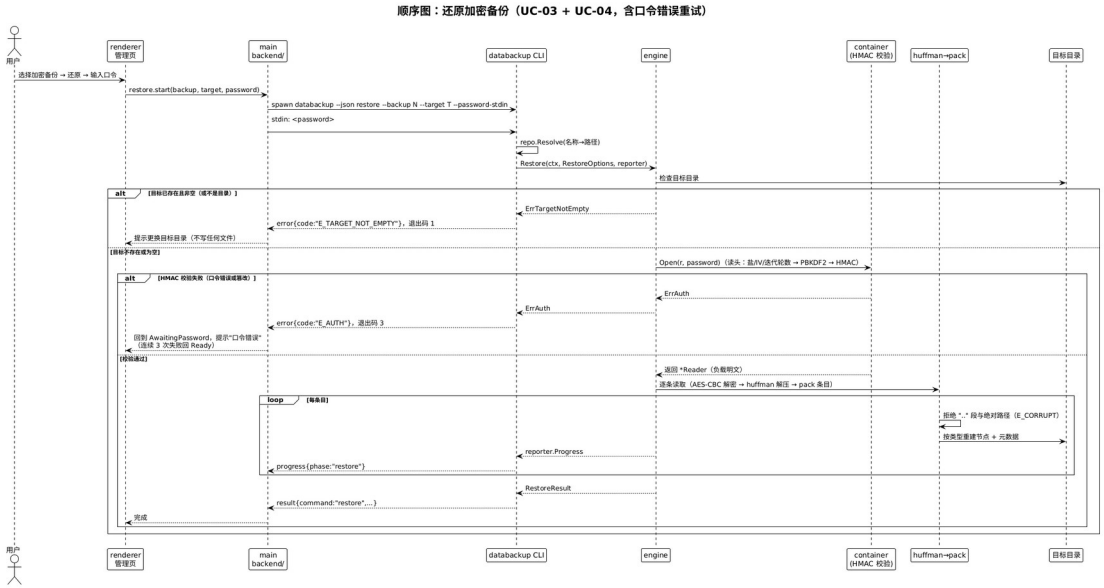
1. renderer 收集参数与口令，经 preload 调 `backup.start`；main 以 `spawn`（无 shell）启动 `databackup --json backup --encrypt aes256 --password-stdin …`，口令写入 stdin。
2. cli 解析校验参数后调 `engine.Backup`：先 scan（walk+filter 统计总量），再以 `container.NewWriter(w, Header, password)` 写出头部，逐条 `pack.WriteHeader/Write` → huffman 压缩 → AES-CBC 加密，顺序写 `<name>.dbk.partial`；progress 事件节流回传。
3. 全部写完后依次 `pack.Writer.Close`、`huffman.Writer.Close`，再 `container.Writer.Finish(Summary)` 写尾部统计与 HMAC 校验值，`.partial` 经硬链接原子发布为 `.dbk`（防并发同名覆盖）；`result` 事件输出统计；进程 exit 0，客户端回填备份列表。
4. 失败/取消：删除 `.partial`，输出对应 `error.code` 与非零退出码。

4.1.2 场景：还原加密备份（含口令校验）

4.1.2.1 场景描述

用户对加密备份发起还原；后端先校验 HMAC（口令错误 → `E_AUTH`，客户端回到口令输入态，连续 3 次失败回到就绪态），校验通过后解密解压逐条重建（UC-03/04，FR-17）。

4.1.2.2 顺序图



（源文件 `diagrams/seq-restore-encrypted.puml`。）

4.1.2.3 流程说明

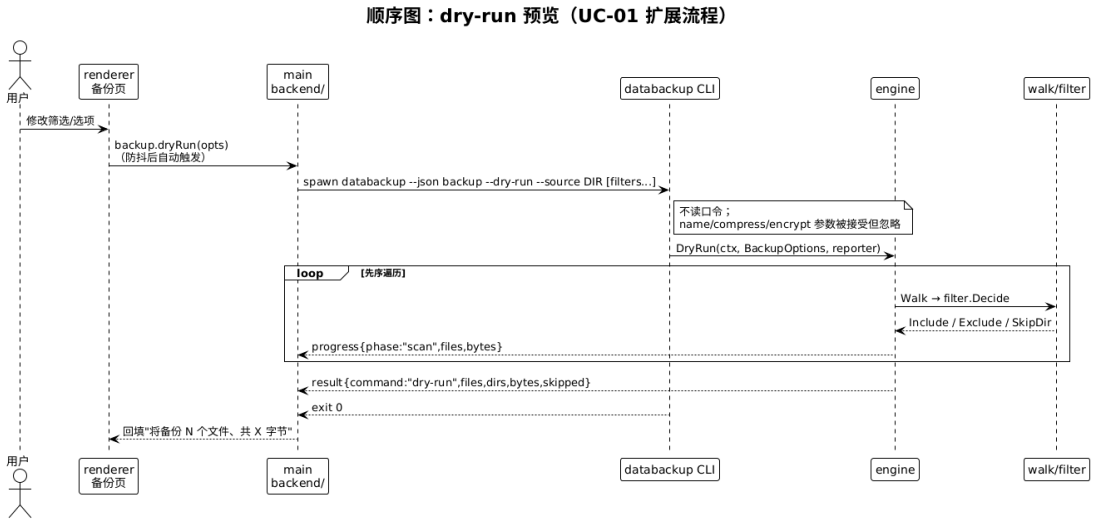
1. `cli` 先经 `repo.Resolve` 把名称解析为 `.dbk` 路径，再调 `engine.Restore`。
2. 目标目录在开始前检查：已存在且非空（或不是目录）→ `ErrTargetNotEmpty`（E_TARGET_NOT_EMPTY、退出码 1），不写任何文件。
3. `container.Open(r, password)` 读头部（盐/IV/迭代轮数）→ PBKDF2 派生密钥 → 完整读一遍做 HMAC 校验；未给口令返回 `ErrPasswordRequired`（E_PASSWORD_REQUIRED），校验失败返回 `ErrAuth`（E_AUTH），退出码均为 3，且不写任何文件。
4. 校验通过后逐条读取：AES-CBC 解密 → huffman 解压 → pack 条目；拒绝 `../`绝对路径条目（E_CORRUPT，路径穿越防护）；按类型创建节点（特殊节点用 `fsmeta.CreateNode`）并以 `fsmeta.ApplyMetadata` 恢复元数据（目录在其子项全部还原后再设置）。

4.1.3 场景：dry-run 预览

4.1.3.1 场景描述

用户修改筛选条件时，客户端防抖触发 `backup --dry-run`，只扫描统计不写文件、不读口令，用于表单下方的预览（UC-01 扩展流程）。

4.1.3.2 顺序图



（源文件 `diagrams/seq-dry-run.puml`。）

4.1.3.3 流程说明

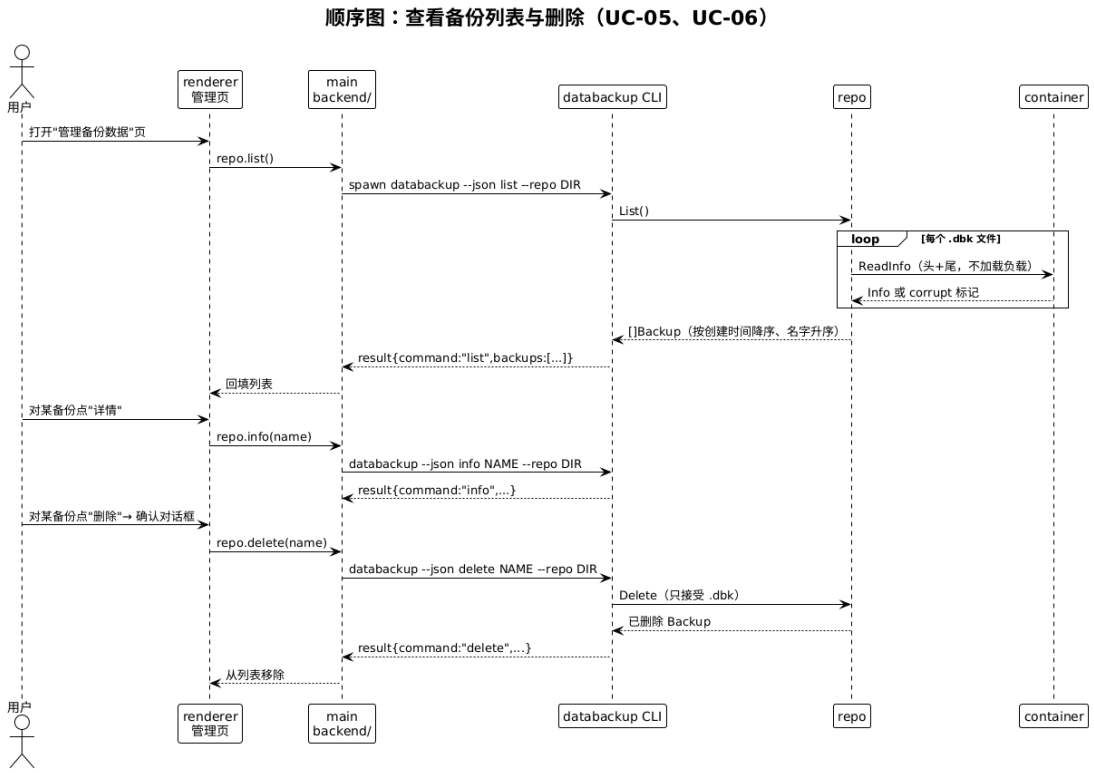
1. `cli` 接受 name/compress/encrypt 参数但忽略（契约 §2.1），不读口令。
2. `engine.DryRun` 仅执行 walk+filter：排除目录→SkipDir，目录只看 types，非目录过全部条件；progress{phase:"scan"} 回传。
3. 返回 `result{command:"dry-run",files,dirs,bytes,skipped}`，客户端展示预计规模。

4.1.4 场景：查看备份列表与删除

4.1.4.1 场景描述

"管理备份数据"页加载时调用 `list` 扫描仓库（无数据库，读容器头尾），用户可查看 `info` 详情或经确认对话框 `delete`（UC-05/06，FR-14）。

4.1.4.2 顺序图



（源文件 `diagrams/seq-list-delete.puml`。）

4.1.4.3 流程说明

1. `repo.List` 扫描 `.dbk`（忽略 `.partial` 与其他文件），逐文件 `container.ReadInfo` 读头尾；无法解析的文件以 `corrupt=true` 列出；按创建时间降序、名字升序。
2. `info` 读取单个备份的头尾元数据（不校验完整负载）。
3. `delete` 只接受 `.dbk`（显式路径必须 `.dbk` 结尾，否则 E_USAGE），客户端先二次确认。

4.2 系统状态

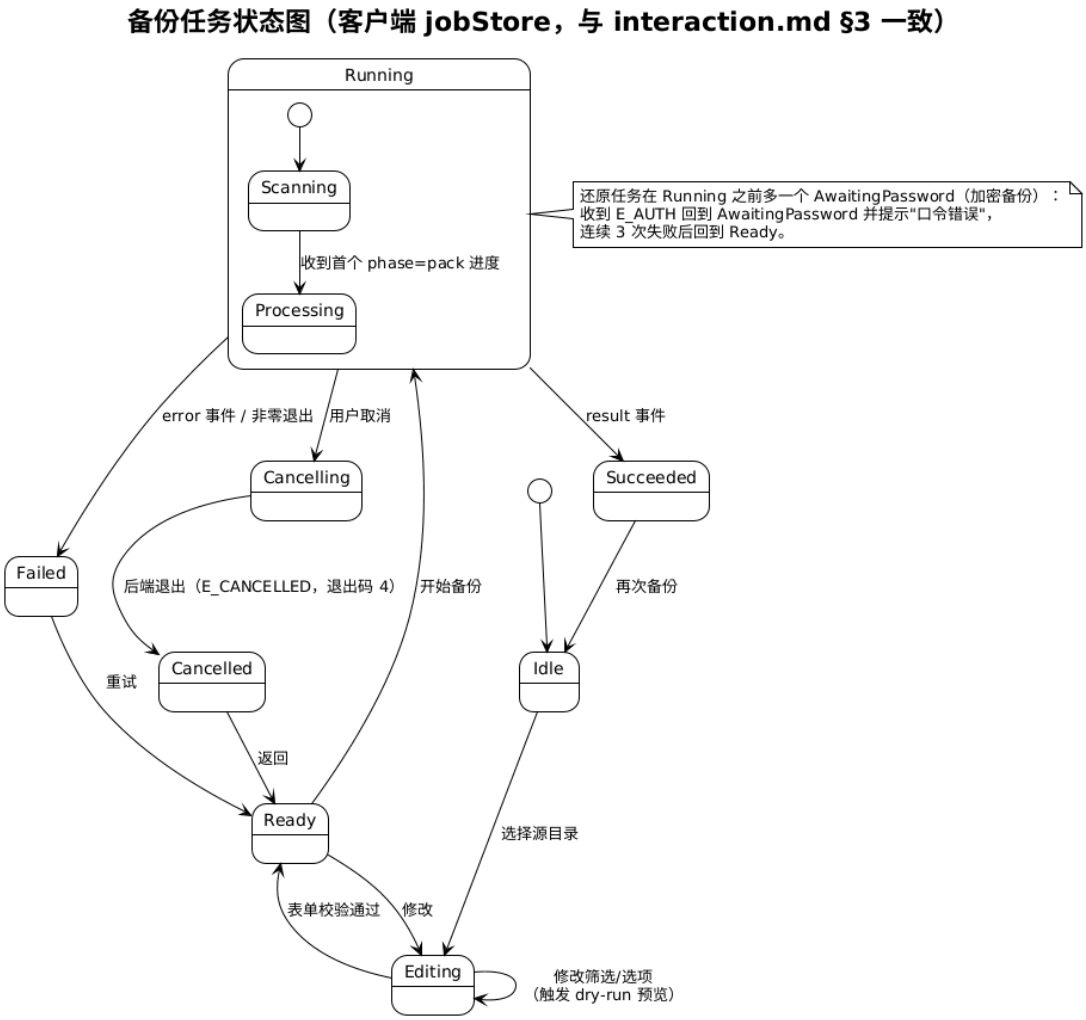
4.2.1 备份任务状态切换

4.2.1.1 状态描述

状态	含义
Idle	初始：未选源目录
Editing	编辑参数中；每次修改触发 dry-run 预览
Ready	表单校验通过，可开始
Running（Scanning/ Processing）	后端执行中；scan 进度 →Scanning，首个 pack 进度

状态	含义
	→Processing
Cancelling	用户已请求取消，等待后端退出
Succeeded / Failed / Cancelled	结果态：result 事件 / error 或非零退出 / E_CANCELLED

4.2.1.2 状态转换图



（源文件 `diagrams/state-backup-job.puml`，与 `../client/interaction.md` §3 一致。）

4.2.1.3 状态图说明

- 取消发 SIGTERM（3 秒未退出升级 SIGKILL）→ctx 取消→后端删 `.partial`、以退出码 4 结束，状态机进 Cancelled；Failed/Cancelled 均可回到 Ready 重试。
- 还原任务在 Running 之前多一个 `AwaitingPassword`（加密备份）：`E_AUTH` 回到该态并提示"口令错误"，连续 3 次失败回到 Ready。

5. 数据库设计（如有）

本系统不使用数据库（ADR-005）。持久化只有三类文件：`.dbk` 备份文件、`config.json` 配置文件、源/还原目录本身。备份仓库的"列表查询"由扫描目录下 `.dbk` 文件并读取容器头尾完成——元数据冗余在容器内，无需索引。

`.dbk` 容器格式（`container` 包，字节级字段表见 `../architecture/container-format.md`）。整数一律小端序，结构为 头部 ‖ 负载 ‖ 尾部 ‖ 校验值：

区段	字段	说明
头部（明文）	magic	格式魔数 `DBAK`
	version	格式版本（当前 `FormatVersion=1`）
	compression	负载压缩算法（none/huffman）
	encryption	负载加密算法（none/aes256-cbc）
	createdAt	备份创建时间（秒精度）
	source	备份时源目录绝对路径
	iterations, salt, IV	PBKDF2 迭代轮数（默认 100000，上限 1000000）、16 字节盐、16 字节 CBC 初始向量（仅加密，`crypto/rand` 生成）；PBKDF2 输出 64 字节，前 32 字节为 AES 密钥、后 32 字节为 HMAC 密钥
负载	MyPack 流	`pack.Writer` 产出的归档字节流，可选 Huffman 压缩、可选 AES-256-CBC 加密
尾部（明文，40 字节）	magic, payloadLen, files, dirs, bytes	尾部魔数 `DBKE`、负载长度、备份统计（同 backup 结果口径）

区段	字段	说明
校验值	HMAC-SHA256 或 SHA-256	加密：HMAC(头部 ‖ 密文 ‖ 尾部)，encrypt-then-MAC；未加密：SHA-256(头部 ‖ 负载 ‖ 尾部)

MyPack 负载内部：每条目为 512 字节头记录（路径、类型、权限、属主、时间戳、尺寸、链接目标、设备号、校验和等）+ 数据块（512 对齐），长文件名使用多头块，归档以零块结束。头块为定长小端二进制字段：魔数 `MYPK`、版本、类型码（沿用 tar 的 `0`~`6`，套接字扩展为 `7`）、CRC-32 校验和、权限/属主/设备号、长度、纳秒级 mtime/atime/ctime/btime，以及 408 字节内联名字区（放不下时溢出到扩展块）。读取时拒绝绝对路径、`..` 段和位于已出现软链接之下的路径。完整字段表见 `../architecture/mypack-format.md`。

配置存储：`$XDG_CONFIG_HOME/databackup/config.json`（未设置 XDG 时 `~/.config/...`），JSON 对象，当前仅 `repo`（绝对路径）一键；`Save` 走同目录临时文件原子改名。